Day 10 =>

Set Interface =>

↳ it extends Collection interface and represents a collection that can not contain duplicate elements. It model the mathematical set abstraction ensuring each element appears at most once

Set Interface Implementation

1. HashSet
2. LinkedHashSet
3. TreeSet

Set is a collection which contains no duplicate elements

It contains no pair e1 and e2 such that e1.equals(e2)

and at most one null element

What is hashing ?

1. It is if we want to search a record from data

   i.e to Roll no. out of 1 lakh student then it is very expensive & it takes O(n) complexity

(2) if we have a particular locker i.e Roll no 60 has locker no. 60 then it is efficient and easy to find

but it is not good for large amount of data as SC => O(n)

(3) If we have 10 lockers and 60 students to store in it

(0 to 9)

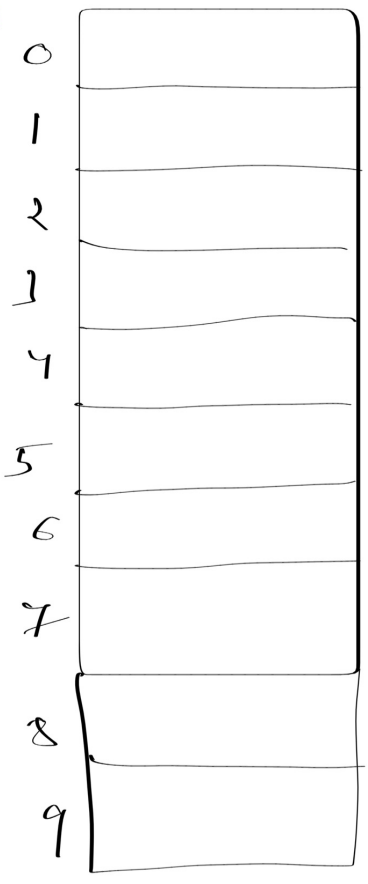
Roll No. % 10 → Hash function

60 % 10 = 0

23 % 10 = 3

55 % 10 = 5

→ Hashcode



It convert any data into small int value → known as hashing

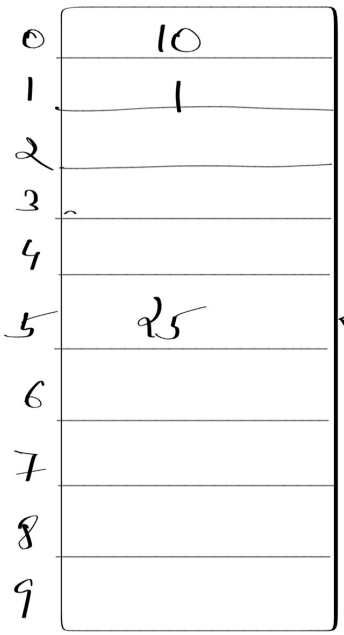
known as hashcode

and these hashcode decide where that object / key will be stored

If there is already object is present and another object come then it is called collision

Ex => keys : 25, 1, 10, 15, 42, 67, 75

Hash function = value % 10



25 % 10 = 5

1 % 10 = 1

10 % 10 = 0

15 % 10 = 5

There are two techniques to handle hashing

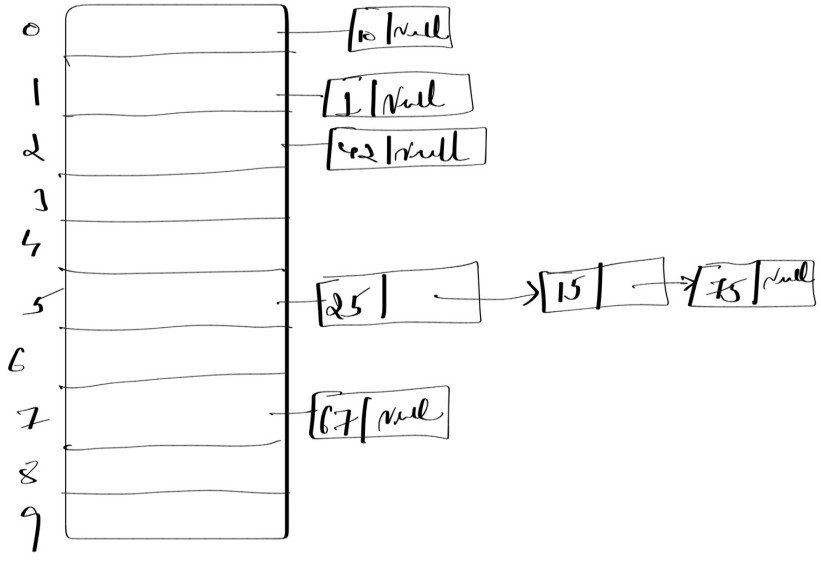
① Seperate chaining

② Open addressing

① Seperate chaining

keys : 25, 1, 10, 15, 42, 67, 75

Hash function = value % 10



Note → Java uses this seperate chaining technique

(2) Open Addressing ⇒ There are three set of rules by which keys are filled or extracted

(i) Linear probing
(ii) Quadratic probing
(iii) Double Hashing

(i) Linear Probing ⇒

keys: 25, 1, 10, 15, 42, 67, 75, 89, 95

15 % 10 = 5

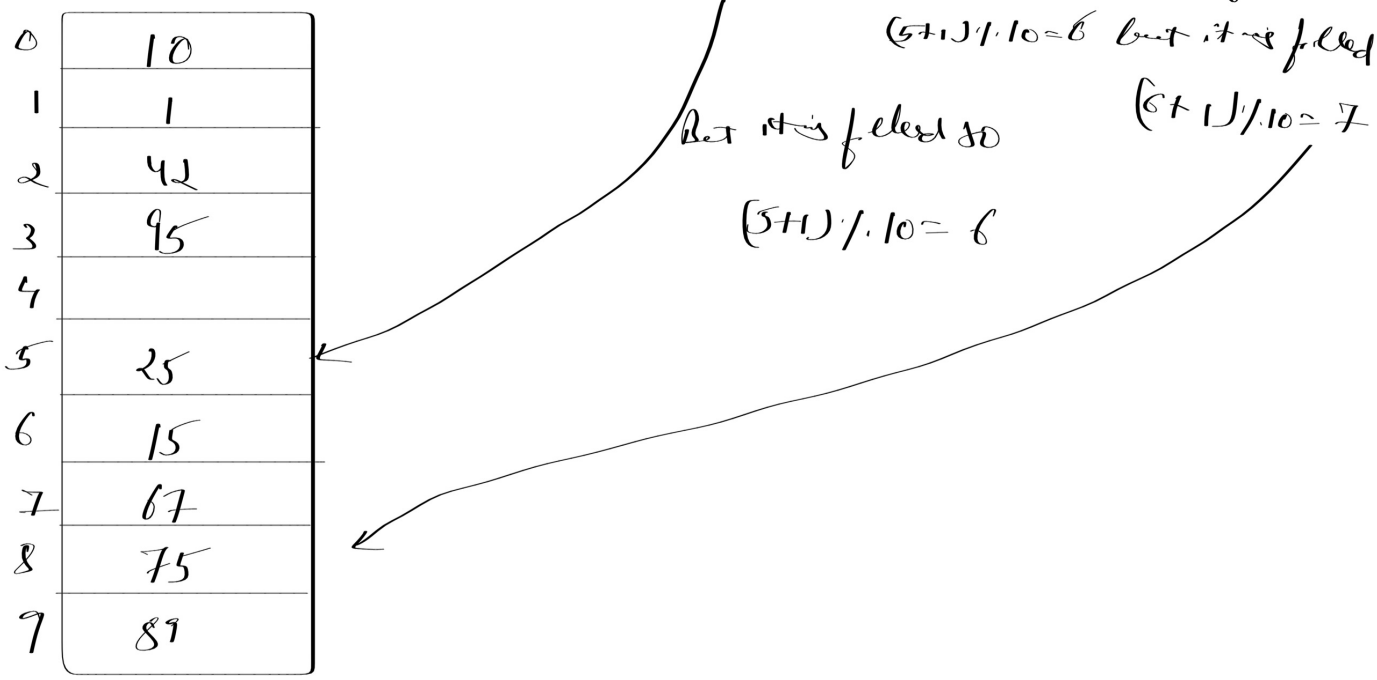
75 % 10 = 5 but it is filled

(5+1) % 10 = 6 but it is filled

(6+1) % 10 = 7

But it is filled so

(5+1) % 10 = 6



Disadvantage of linear probing is that searching for an element take a long time. as we have to search linearly

(ii) Quadratic probing ⇒ 25, 1, 10, 15, 42, 67, 73, 89, 95

we have to go with $(index + i^2) \% 10$

where $i = 0, 1, 2, \ldots$

$5 \% 10 = 5$ (index)

$(5+0^2) \% 10 = 5$ filled

$(5+1^2) \% 10 = 6$

0	10
1	1
2	42
3	89
4	95
5	25
6	13
7	67
8	
9	73

$73 \% 10 = 5$

$(5+0^2) \% 10 = 5$ (filled)

$(5+1^2) \% 10 = 6$ (filled)

$(5+2^2) \% 10 = 9$

$89 \% 10 = 9$

$(9+0^2) \% 10 = 9$ (filled)

$(9+1^2) \% 10 = 0$ (filled)

$(9+2^2) \% 10 = 3$

$95 \% 10 = 5$

$(5+0^2) \% 10 = 5$ (filled)

$(5+1^2) \% 10 = 6$ (filled)

$(5+2^2) \% 10 = 9$ (filled)

$(5+3^2) \% 10 = 4$

(iii) Double Hashing ⇒ In this we use double hash functions

If hash function if it index is filled then use another hash function

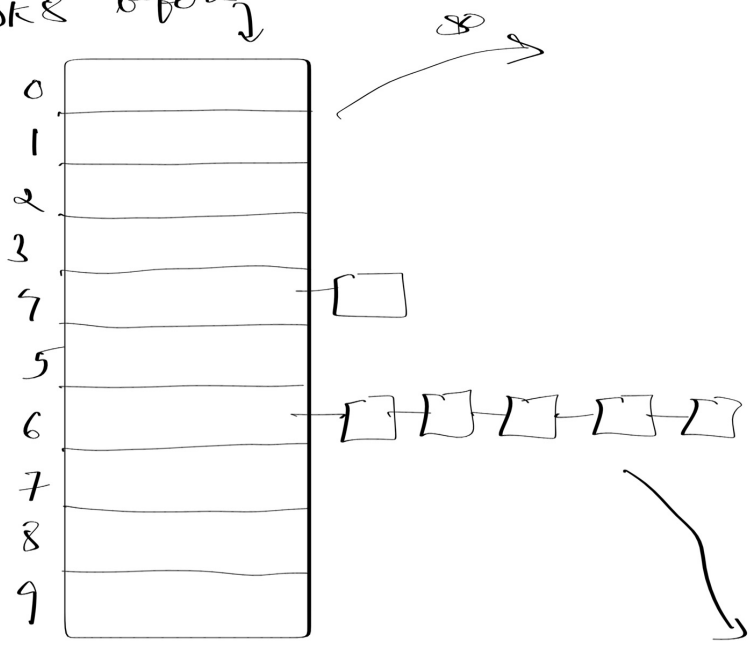
Note => hash function should be good

satisfies two properties

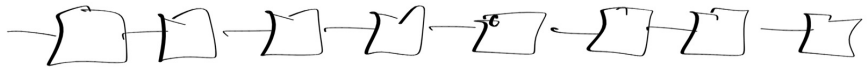
① Quickly locate

② uniformly disperse elements

JDK8 before



but after JDK8 if Chaining ≥ 8 and default bucket size (initially) is 16 (not 8) / array size ≥ 64

then 

this linked list is replaced by red black trees (RBT)

a balanced tree which has searching TC O(log n)

from TC O(n) in linked list

Note => In resizing array / bucket load factor plays an important role, in hashmap default load factor is 0.75 i.e 75% (if 75% of array / bucket is filled then it is resize)

HashSet

- implementation of Set interface
- stores unique elements
- fast insertion, deletion and look up using hashing
- internally use HashMap
- at most one null is allowed

note => HashSet internally used HashMap in which elements are stored in key value pairs but why does HashSet store only key ?

Actually in HashSet for every key there is a dummy value

i.e <k, Present>

methods

① add(E e) =>

```
HashSet<Integer> h1 = new HashSet<>();
h1.add(1);
```

→ Internally it uses hashing

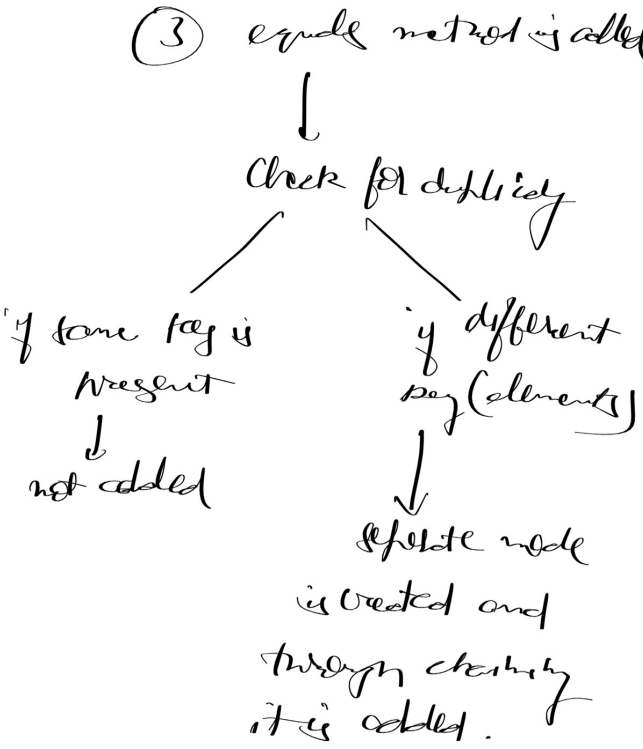
① hash(key)

② hashcode

↓

find bucket where to store e

- if bucket is empty then put object e
- if bucket is not empty ↓


(3) equals method is called
Check for duplicacy
if same tag is present
not added
if different tag (elements)
Update node is created and through checking it is added.